Guía Explicada Línea por Línea: Proyecto "Riviera Express"

Este documento explica **exactamente qué hace cada archivo y cada línea de código** del proyecto para que cualquier integrante del equipo pueda exponerlo y responder preguntas del profesor.

1. Resumen Rápido: ¿Cómo cumplimos lo que pidió el profesor?

Concepto pedido	¿Dónde está en nuestro código?
1 Archivo = 1 Clase	`Envio.py`, `Paquete.py`, `Documento.py`, `Repartidor.py`, `HojaRuta.py`
1 main = Objetos	`main.py` importa las clases con `from Archivo import Clase` y crea los objetos
Constructores y Destructores	Todas las clases tienen `def __init__` y `def __del__`
Herencia	`Paquete` y `Documento` heredan de la clase padre `Envio`
Encapsulamiento 1 (get/set)	En `Envio.py` (`get_peso`, `set_peso`, `get_distancia`, `set_distancia`) y `Repartidor.py`
Encapsulamiento 2 (@property)	En `Paquete.py` (`@property def volumen`) y `HojaRuta.py` (`@property def repartidor`)
Sobrecarga de operadores	En `Envio.py` con `def __add__` para sumar dos envíos usando `+` (`paquete_1 + paquete_2`)
Sin try/except	Validamos que no haya números negativos usando solo `if nuevo_peso > 0:`

2. Explicación Archivo por Archivo (Línea por Línea)

Archivo 1: Envio.py (La Clase Padre / Superclase)

Representa cualquier envío de la paquetería. De aquí van a heredar `Paquete` y `Documento`.

```
class Envio:
    def __init__(self, codigo, peso, distancia):
        self.codigo = codigo          # Público (+)
        self.__peso = peso            # Privado (-)
        self.__distancia = distancia  # Privado (-)

    def __del__(self):
        print(f"Envío {self.codigo} eliminado")

    # Encapsulamiento 1: get y set
    def get_peso(self):
        return self.__peso

    def set_peso(self, nuevo_peso):
        if nuevo_peso > 0:
            self.__peso = nuevo_peso
        else:
            print("El peso no puede ser negativo")

    def get_distancia(self):
        return self.__distancia

    def set_distancia(self, nueva_distancia):
        if nueva_distancia > 0:
            self.__distancia = nueva_distancia
        else:
            print("La distancia no puede ser negativa")

    def calcular_costo(self):
        return (self.__peso * 10) + (self.__distancia * 5)

    # Sobrecarga del operador '+' para sumar costos de dos envíos
    def __add__(self, otro_envio):
        return self.calcular_costo() + otro_envio.calcular_costo()
```

- **`def __init__(self, codigo, peso, distancia):`** Constructor que se ejecuta al crear el envío.
- **`self.codigo = codigo`**: Atributo **público (+)**. Se puede leer directo sin getter.
- **`self.__peso`** y **`self.__distancia`**: Atributos **privados (-)** con doble guion bajo `__` para protegerlos de valores negativos.
- **`def __del__(self):`** Destructor que avisa cuando el objeto se borra de la memoria.
- **`get_peso`** y **`set_peso`**: Encapsulamiento tradicional. El `set_peso` usa `if nuevo_peso > 0:` para impedir pesos negativos.
- **`calcular_costo(self)`**: Calcula la tarifa base: **$10 por kg** + **$5 por km**.
- **`def __add__(self, otro_envio):`** Sobrecarga del signo `+`. Suma el costo de dos envíos cuando hacemos `paquete_1 + paquete_2`.

Archivo 2: Paquete.py (Subclase con Herencia y @property)

Representa una caja o paquete que, además de peso y distancia, tiene **volumen**.

```
from Envio import Envio


class Paquete(Envio):
    def __init__(self, codigo, peso, distancia, volumen):
        super().__init__(codigo, peso, distancia)
        self.__volumen = volumen  # Privado (-)

    def __del__(self):
        print(f"Paquete {self.codigo} eliminado")

    # Encapsulamiento 2: @property
    @property
    def volumen(self):
        return self.__volumen

    @volumen.setter
    def volumen(self, nuevo_volumen):
        if nuevo_volumen > 0:
            self.__volumen = nuevo_volumen
        else:
            print("El volumen no puede ser negativo")

    def calcular_costo(self):
        return super().calcular_costo() + (self.__volumen * 20)
```

- **`class Paquete(Envio):`** Hereda todos los atributos y métodos de `Envio`.
- **`super().__init__(codigo, peso, distancia)`**: Llama al constructor de la clase padre `Envio`.
- **`@property`** y **`@volumen.setter`**: Segunda forma de encapsulamiento. Permite usar `paquete_1.volumen = 3` validando que no sea negativo.
- **`calcular_costo(self)`**: Suma el costo base del padre (`super().calcular_costo()`) más **$20 por unidad de volumen**.

Archivo 3: Documento.py (Segunda Subclase con Herencia)

```
from Envio import Envio


class Documento(Envio):
    def __init__(self, codigo, peso, distancia, urgente):
        super().__init__(codigo, peso, distancia)
        self.urgente = urgente  # Público (+)

    def __del__(self):
        print(f"Documento {self.codigo} eliminado")

    def calcular_costo(self):
        costo = super().calcular_costo()
        if self.urgente:
            return costo + 30
        return costo
```

- Hereda de `Envio`. Si `self.urgente` es `True`, suma **$30 extra** al costo base.

Archivo 4: Repartidor.py (Clase del Repartidor Responsable)

```
class Repartidor:
    def __init__(self, id_repartidor, nombre, vehiculo):
        self.nombre = nombre                # Público (+)
        self.vehiculo = vehiculo            # Público (+)
        self.__id_repartidor = id_repartidor # Privado (-)

    def __del__(self):
        print(f"Repartidor {self.nombre} eliminado")

    def get_id_repartidor(self):
        return self.__id_repartidor

    def set_id_repartidor(self, nuevo_id):
        self.__id_repartidor = nuevo_id
```

- Deja `nombre` y `vehiculo` públicos (+) para leerlos fácilmente y protege `__id_repartidor` como privado (-) con `get` y `set`.

Archivo 5: HojaRuta.py (Agrupa Envíos y Asigna Repartidor)

```
class HojaRuta:
    def __init__(self, folio, zona):
        self.folio = folio        # Público (+)
        self.zona = zona          # Público (+)
        self.__repartidor = None  # Privado (-)
        self.__envios = []        # Privado (-)

    def __del__(self):
        print(f"Hoja de ruta {self.folio} eliminada")

    @property
    def repartidor(self):
        return self.__repartidor

    @repartidor.setter
    def repartidor(self, nuevo_repartidor):
        self.__repartidor = nuevo_repartidor

    def agregar_envio(self, envio):
        self.__envios.append(envio)

    def mostrar_ruta(self):
        print(f"--- Hoja de Ruta: {self.folio} ({self.zona}) ---")
        print(f"Repartidor: {self.__repartidor.nombre} - Vehículo: {self.__repartidor.vehic
        print(f"Cantidad de envíos: {len(self.__envios)}")
```

- **`self.__envios = []`**: Lista donde se agrupan varios paquetes y documentos con `agregar_envio()`.
- **`@property def repartidor`**: Permite asignar el repartidor responsable a la hoja de ruta.

Archivo 6: main.py (Creación de Objetos y Pruebas)

```
from Paquete import Paquete
from Documento import Documento
from Repartidor import Repartidor
from HojaRuta import HojaRuta

# 1. Crear objetos (Herencia de Envio)
paquete_1 = Paquete("PAQ-01", 5, 10, 2)
paquete_2 = Paquete("PAQ-02", 3, 8, 1)
doc_1 = Documento("DOC-01", 1, 4, True)

print(f"Costo Paquete 1: ${paquete_1.calcular_costo()}")
print(f"Costo Paquete 2: ${paquete_2.calcular_costo()}")
print(f"Costo Documento 1: ${doc_1.calcular_costo()}")

# 2. Probar encapsulamiento y validación de negativos (get/set y @property)
paquete_1.set_peso(-5)     # No permite negativo (get/set)
paquete_1.volumen = -3     # No permite negativo (@property)

paquete_1.set_peso(6)      # Cambio válido con set
paquete_1.volumen = 3      # Cambio válido con @property
print(f"Nuevo costo Paquete 1: ${paquete_1.calcular_costo()}")

# 3. Sobrecarga de operador '+' (__add__)
suma_envios = paquete_1 + paquete_2
print(f"Suma de Paquete 1 y Paquete 2: ${suma_envios}")

# 4. Hoja de ruta y repartidor responsable
repartidor_1 = Repartidor("REP-01", "Carlos", "Moto")
ruta_1 = HojaRuta("RUTA-100", "Centro")

ruta_1.repartidor = repartidor_1
ruta_1.agregar_envio(paquete_1)
ruta_1.agregar_envio(paquete_2)
ruta_1.agregar_envio(doc_1)

ruta_1.mostrar_ruta()

# 5. Destructores (__del__)
del paquete_1
del paquete_2
del doc_1
del ruta_1
del repartidor_1
```

3. Salida Exacta en Consola al Ejecutar main.py

```
Costo Paquete 1: $140
Costo Paquete 2: $90
Costo Documento 1: $60
El peso no puede ser negativo
El volumen no puede ser negativo
Nuevo costo Paquete 1: $170
Suma de Paquete 1 y Paquete 2: $260
--- Hoja de Ruta: RUTA-100 (Centro) ---
Repartidor: Carlos - Vehículo: Moto
Cantidad de envíos: 3
Hoja de ruta RUTA-100 eliminada
Documento DOC-01 eliminado
Paquete PAQ-02 eliminado
Paquete PAQ-01 eliminado
Repartidor Carlos eliminado
```